

Analysis of Numerical Algorithms

From Root-Finding to Fractals

Author: Jaya Patel

Institution: IIT Delhi

Department: Mathematics and Computing

Date: June 19, 2026

Abstract

Numerical methods form the computational backbone of modern science and engineering. This paper presents an analysis of classical numerical algorithms — root-finding methods (Bisection and Secant), polynomial interpolation techniques (Lagrange, Newton Divided Differences, and Cubic Hermite), and Newton's Method for nonlinear equations. Each algorithm is accompanied by its mathematical derivation and a Python implementation. The paper culminates in a visually striking application: Newton Fractals — where the geometry of complex roots gives rise to infinite, self-similar beauty at the boundary of convergence basins. We also outline directions for future work, including dynamic, interactive fractal visualizations.

1 Introduction to Numerical Algorithms

Mathematics, at its purest, deals in exact solutions. But the real world rarely cooperates. Physical systems, financial models, and engineering structures generate equations that resist closed-form answers — transcendental functions, nonlinear systems, and differential equations that defy algebraic manipulation. This is precisely where **numerical algorithms** step in, transforming intractable problems into sequences of manageable computations that converge, step by step, toward a reliable answer.

A numerical algorithm is a finite, deterministic procedure that produces an approximate solution to a mathematical problem within a specified tolerance. Unlike symbolic algebra, which seeks exact answers, numerical methods embrace approximation as a feature rather than a flaw — trading perfect precision for practical computability.

1.1 Motivation

One of the primary motivations for this project is the deep connection between Newton Fractals and loss landscapes in machine learning. Training a neural network is essentially Newton's Method operating in high dimensions — finding where the gradient is zero. Newton fractals visually demonstrate why optimization can get stuck or converge to unexpected solutions depending on initialization. In this sense, the fractal *is* the loss landscape, made visible.

2 Root-Finding Methods

A *root* of a function f is a value x^* such that $f(x^*) = 0$. Root-finding is one of the oldest and most fundamental problems in numerical analysis. We examine two methods: the guaranteed Bisection Method and the faster Secant Method.

2.1 The Bisection Method

The Bisection Method rests on the **Intermediate Value Theorem**: if f is continuous on $[a, b]$ and $f(a) \cdot f(b) < 0$, then a root exists in (a, b) . The algorithm repeatedly halves the interval, always keeping the half that contains the sign change. It is widely used in **finance** to compute the Internal Rate of Return (IRR), and in **engineering** to find the load at which a structure reaches its stress limit.

Key formula — the midpoint at each step:

$$c_n = \frac{a_n + b_n}{2}$$

After n iterations, the error is bounded by:

$$|x^* - c_n| \leq \frac{b - a}{2^n}$$

It is *slow but guaranteed to converge* provided the initial bracket $[a, b]$ is valid.

2.1.1 Python Implementation

```
1 def my_function(x):
2     return x**2 - 4*x - 10
3
4 def bisection_method(func, a, b, tol=1e-6, max_iter=100):
5     if func(a) * func(b) >= 0:
6         raise ValueError("The Bisection Method requires that "
7                             "f(a) and f(b) have opposite signs.")
8     for _ in range(max_iter):
9         c = (a + b) / 2
10        fc = func(c)
11
12        if abs(fc) < tol or (b - a) / 2 < tol:
13            return c
14
15        if func(a) * fc < 0:
16            b = c
17        else:
18            a = c
19    return c
20
21 root = bisection_method(my_function, -2.0, 0.0, tol=1e-4)
22 print(f"Root: {root}")
```

Listing 1: Bisection Method

2.2 The Secant Method

Instead of halving intervals, the Secant Method draws a line through two recent approximations and takes its x-intercept as the next estimate. It requires no derivative and is significantly faster than Bisection in practice, though it may diverge with poor initial guesses. It is commonly used in **electrical engineering** to solve nonlinear circuit equations (e.g. diode models), and in **scientific computing** to find roots of complex polynomials where derivatives are costly to evaluate.

Key formula:

$$x_{n+1} = x_n - f(x_n) \cdot \frac{x_n - x_{n-1}}{f(x_n) - f(x_{n-1})}$$

2.2.1 Python Implementation

```
1 def secant(f, x0, x1, eps, itmax):
2     itnum = 1
3
4     while True:
5         x2 = x1 - (f(x1) * (x1 - x0)) / (f(x1) - f(x0))
6
7         if abs(x2 - x1) <= eps:
8             return x2, 0
9
10        if itnum == itmax:
11            return x2, 1
12        itnum += 1
13        x0 = x1
```

```

14         x1=x2
15
16 f = lambda x: x**6-x- 1
17
18 root, ier = secant(f,x0=1,x1=2.0,eps=1e-7, itmax=100)
19
20 if ier == 0:
21     f_root=f(root)
22     print(f"Converged! Root = {root},{f_root}")
23 elif ier == 1:
24     print(f"Max iterations reached. Best estimate = {root}")

```

Listing 2: Secant Method

3 Polynomial Interpolation

Interpolation is the art of reading between data points. Given $n+1$ nodes $(x_0, y_0), \dots, (x_n, y_n)$, an interpolating polynomial passes through every point exactly. By the *Weierstrass Approximation Theorem*, there is exactly one polynomial of degree $\leq n$ through $n+1$ distinct points. Interpolation is a mathematical technique used in the **stock market** to estimate unknown values between two known data points. It is primarily used to build continuous yield curves, smooth out missing data, and price complex financial derivatives when exact market quotes are unavailable

3.1 Lagrange Interpolation

Lagrange interpolation builds the polynomial from basis functions, each equal to 1 at one node and 0 at all others. It is used in **numerical integration** (as the basis for Newton-Cotes formulas) and in **signal processing** for reconstructing a continuous signal from discrete samples.

Lagrange basis polynomial:

$$L_k(x) = \prod_{\substack{j=0 \\ j \neq k}}^n \frac{x - x_j}{x_k - x_j}$$

Interpolating polynomial:

$$P(x) = \sum_{k=0}^n y_k L_k(x)$$

Python Implementation:

```

1 def lagrange_interpolation(x, y, t):
2     n = len(x)
3     p= 0.0
4     for i in range(n):
5         L = 1.0
6         for j in range(n):
7             if i != j:
8                 L *= (t - x[j]) / (x[i] - x[j])
9     p += y[i] * L

```

```

10     return p
11 #example:
12 t=[2,3,8]
13 x=[1,3,5,7,8]
14 y=[2,1,5,3,9]
15 for j in t:
16     y2=lagrange_interpolation(x, y, j)
17     print(f"value at {j} = {y2}")

```

Listing 3: Lagrange Interpolation

Figure 1 illustrates Lagrange interpolation applied to $f(x) = \sin(x)$ over $[0, \pi]$ using three randomly chosen nodes. Despite using only a handful of points, the interpolating polynomial $P(x)$ closely tracks the true sine curve — demonstrating how Lagrange interpolation can reconstruct smooth, continuous functions from minimal sampled data.

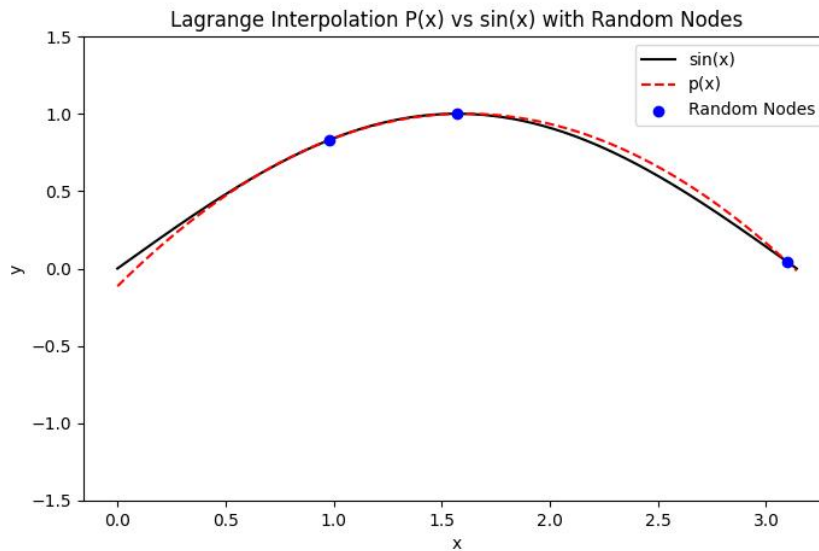


Figure 1: Lagrange interpolation $P(x)$ approximating $\sin(x)$ with 3 random nodes.

3.2 Newton's Divided Difference Interpolation

Newton's Divided Difference method builds the same unique polynomial as Lagrange but in a form that is easy to extend when new data arrives — only one new column of differences is needed. It is preferred in **real-time data processing** where measurements arrive sequentially, and in **numerical differentiation** where divided differences directly approximate derivatives.

Divided differences (defined recursively):

$$f[x_k] = y_k, \quad f[x_k, \dots, x_{k+j}] = \frac{f[x_{k+1}, \dots, x_{k+j}] - f[x_k, \dots, x_{k+j-1}]}{x_{k+j} - x_k}$$

Newton's forward form:

$$P(x) = f[x_0] + f[x_0, x_1](x - x_0) + f[x_0, x_1, x_2](x - x_0)(x - x_1) + \dots$$

Python Implementation

```

1 def divdif(f_values, x):
2     d = f_values
3     n = len(x) - 1
4     for i in range(1, n + 1):
5         for j in range(n, i - 1, -1):
6             d[j] = (d[j] - d[j-1]) / (x[j] - x[j-i])
7     return d
8
9 def interp(d, x, t):
10    n = len(d) - 1
11    p = d[n]
12    for i in range(n - 1, -1, -1):
13        p = d[i] + (t - x[i]) * p
14    return p
15
16 x = [0, 1, 2, 3]
17 f = [1, 2, 5, 10]
18
19 d = divdif(f, x)
20 t = 1.5
21 print(interp(d, x, t))

```

Listing 4: Newton's Divided Difference Interpolation

3.3 Cubic Hermite Interpolation

Hermite interpolation matches both *function values* and *derivatives* at each node, producing C^1 -smooth curves across interval boundaries. It is the foundation of **spline-based animation** in computer graphics (used in Bezier and NURBS curves), and in **robotics** for generating smooth joint trajectories where both position and velocity must be controlled.

Hermite basis functions on $[x_i, x_{i+1}]$, with $t = \frac{x-x_i}{x_{i+1}-x_i}$:

$$\begin{aligned}
 h_{00}(t) &= 2t^3 - 3t^2 + 1, & h_{10}(t) &= t^3 - 2t^2 + t \\
 h_{01}(t) &= -2t^3 + 3t^2, & h_{11}(t) &= t^3 - t^2
 \end{aligned}$$

Interpolant:

$$p(t) = h_{00}f(x_i) + h_{10}(x_{i+1} - x_i)f'(x_i) + h_{01}f(x_{i+1}) + h_{11}(x_{i+1} - x_i)f'(x_{i+1})$$

Python Implementation

```

1 def y(x):
2     y=[]
3     for i in x:
4         f=1/(1+(i*i))
5         y.append(f)
6
7     return y
8
9 def s_i(x,k):
10    i=x[k]
11    f=-(2*i)/((1+i*i)**2)
12    return f

```

```

13
14 def find_interval(x, t):
15     for k in range(len(x) - 1):
16         if x[k] <= t <= x[k+1]:
17             return k
18     return -1
19
20 def P(x,y,s_i,t,k):
21     c1=y[k]
22     c2=s_i(x,k)
23     f_xi_xi1 = (y[k]-y[k+1])/(x[k]-x[k+1])
24     del_xi=x[k+1]-x[k]
25     c4 = (c2 + s_i(x, k+1) - 2 * f_xi_xi1) / (del_xi * del_xi)
26     c3 = ((f_xi_xi1 - c2) / del_xi) - (c4 * del_xi)
27     dx = t - x[k]
28     p = c1 + c2 * dx + c3 * (dx ** 2) + c4 * (dx ** 3)
29     return p
30 x=[1,3,6,8]
31 t=4
32 y=y(x)
33 k=find_interval(x,t)
34 ans=P(x,y,s_i,t,k)
35 print(ans)

```

Listing 5: Cubic Hermite Interpolation

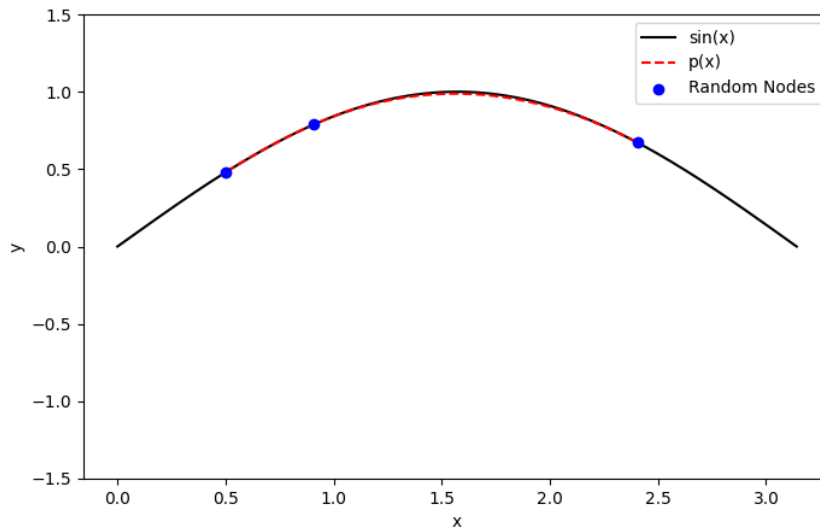


Figure 2: Cubic Hermite Interpolation $P(x)$ approximating $\sin(x)$ with 3 random nodes.

Figure 2 illustrates Cubic Hermite interpolation applied to $f(x) = \sin(x)$ over $[0, \pi]$ using three randomly chosen nodes. Despite using only a handful of points, the interpolating polynomial $P(x)$ closely tracks the true sine curve — demonstrating how Lagrange interpolation can reconstruct smooth, continuous functions from minimal sampled data.

4 Newton's Method

Newton's Method linearizes f at each step. The tangent line at $(x_n, f(x_n))$ crosses zero at:

$$x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n)}$$

It converges *quadratically* near a simple root — the number of correct digits **doubles** with each iteration. To find $\sqrt{2}$, starting from $x_0 = 1$, four iterations give twelve decimal places of accuracy. Newton's Method powers **optimization solvers** in machine learning (quasi-Newton methods like L-BFGS), and is used in **physics simulations** to solve nonlinear systems arising in finite element analysis.

4.1 Python Implementation

```
1 def newton(f, df, x0, eps, itmax):
2
3     itnum = 1
4     while True:
5         denom = df(x0)
6         if denom == 0:
7             return None, 2
8
9         x1 = x0 - f(x0) / denom
10
11         if abs(x1 - x0) <= eps:
12             return x1, 0
13
14         if itnum == itmax:
15             return x1, 1
16
17         itnum += 1
18         x0 = x1
19 f = lambda x: x**6-x- 1
20 df = lambda x: 6*x**5-1
21
22 root, ier = newton(f, df, x0=2.0, eps=1e-7, itmax=100)
23
24 if ier == 0:
25     print(f"Converged! Root = {root}")
26 elif ier == 1:
27     print(f"Max iterations reached. Best estimate = {root}")
28 elif ier == 2:
29     print("Error: derivative was zero at starting point.")
```

Listing 6: Newton's Method

5 Newton Fractals

The Most Beautiful Accident in Numerical Analysis

Ask Newton's Method to find the roots of a complex polynomial — and something extraordinary happens. Color each starting point in the complex plane by which root it

converges to, and you see something no one anticipated: a fractal of infinite, breathtaking complexity.

5.1 Newton's Method in the Complex Plane

Extend Newton's iteration to $\mathbb{C}$. For a polynomial $p(z)$:

$$z_{n+1} = z_n - \frac{p(z_n)}{p'(z_n)}, \quad z_n \in \mathbb{C}$$

For each starting point z_0 on a grid:

1. Run Newton's Method until convergence.
2. Record which root it converges to.
3. Color the pixel at z_0 with that root's color.

The result is a **Newton Fractal**.

5.2 Example: $p(z) = z^3 - 1$

The three roots are:

$$z_1^* = 1, \quad z_2^* = e^{2\pi i/3} = -\frac{1}{2} + \frac{\sqrt{3}}{2}i, \quad z_3^* = e^{4\pi i/3} = -\frac{1}{2} - \frac{\sqrt{3}}{2}i$$

Newton's iteration becomes $z_{n+1} = z_n - \frac{z_n^3 - 1}{3z_n^2}$. Applied to a 1000×1000 grid over $[-2, 2] \times [-2, 2]$, the three colored basins of attraction meet at a boundary of infinite intricacy.

Why Is It a Fractal?

The boundaries between colored regions are *nowhere smooth* — infinitely detailed at every scale. For polynomials of degree ≥ 3 , this boundary is a Julia set with Hausdorff dimension strictly between 1 and 2. Wherever you zoom, the same patterns re-emerge.

5.3 Python Implementation

```
1 import math
2 import matplotlib.pyplot as plt
3 import matplotlib.cm as cm
4 import numpy as np
5
6 class Complex:
7     def __init__(self, real=0.0, imag=0.0):
8         self.real = float(real)
9         self.imag = float(imag)
10    def __add__(self, other):
11        return Complex(self.real + other.real, self.imag + other.imag)
```

```

12     def __sub__(self, other):
13         return Complex(self.real - other.real, self.imag - other.imag
14     )
15     def __mul__(self, other):
16         r = self.real * other.real - self.imag * other.imag
17         i = self.real * other.imag + self.imag * other.real
18         return Complex(r, i)
19     def __truediv__(self, other):
20         denom = other.real**2 + other.imag**2
21         if denom == 0:
22             raise ZeroDivisionError()
23         r = (self.real * other.real + self.imag * other.imag) / denom
24         i = (self.imag * other.real - self.real * other.imag) / denom
25         return Complex(r, i)
26     def __abs__(self):
27         return math.sqrt(self.real**2 + self.imag**2)
28
29 def complex_pow(z, power):
30     if power == 0:
31         return Complex(1.0, 0.0)
32     result = z
33     for _ in range(1, power):
34         result = result * z
35     return result
36
37 def newton(f, df, x0, eps=1e-10, itmax=60):
38     for step in range(itmax):
39         fx = f(x0)
40         dfx = df(x0)
41         if abs(dfx) == 0:
42             return None, itmax
43         x1 = x0 - fx / dfx
44         if abs(x1 - x0) < eps:
45             return x1, step
46         x0 = x1
47     return x0, itmax
48
49 n = 8 #no. of roots
50 def f(z):
51     return complex_pow(z, 8) - Complex(1.0, 0.0) #define f
52
53 def df(z):
54     return Complex(8.0, 0.0) * complex_pow(z, 7) #define df
55
56 np_roots = np.roots([1, 0, 0, 0, 0, 0, 0, 0, -1])
57 roots = [Complex(r.real, r.imag) for r in np_roots]
58
59 def get_root_index(root, tol=1e-2):
60     if root is None:
61         return -1
62     for i, r in enumerate(roots):
63         if abs(root - r) < tol:
64             return i
65     return -1
66
67 def compute_fractal(xmin, xmax, ymin, ymax, width, height, itmax):
68     root_grid = np.zeros((height, width), dtype=int)
69     steps_grid = np.zeros((height, width), dtype=float)
70     for row in range(height):

```

```

69         for col in range(width):
70             real = xmin + (xmax - xmin) * col / width
71             imag = ymin + (ymax - ymin) * row / height
72             z0 = Complex(real, imag)
73             root, steps = newton(f, df, z0, itmax=itmax)
74             idx = get_root_index(root)
75             root_grid[row, col] = idx
76             steps_grid[row, col] = steps
77         return root_grid, steps_grid
78
79 ITMAX = 50
80 xmin, xmax, ymin, ymax = -1.5, 1.5, -1.5, 1.5
81 root_grid, steps_grid = compute_fractal(xmin, xmax, ymin, ymax, width
82                                         =800, height=800, itmax=ITMAX)
83
84 rootColors = cm.rainbow(np.linspace(0, 1, n))[:, :3]
85 height, width = root_grid.shape
86 img = np.zeros((height, width, 3))
87
88 for row in range(height):
89     for col in range(width):
90         root_idx = root_grid[row, col]
91         steps     = steps_grid[row, col]
92         if root_idx == -1:
93             img[row, col] = [0.0, 0.0, 0.0]
94         else:
95             base_color = rootColors[root_idx]
96             norm_steps = steps / ITMAX
97             grad_factor = max(0.35, 1.0 - (norm_steps ** 0.6))
98             img[row, col] = base_color * grad_factor
99
100 plt.figure(figsize=(10, 10))
101 plt.imshow(img, extent=[xmin, xmax, ymin, ymax], origin='lower',
102            interpolation='bilinear')
103 plt.xlabel('Re(z)')
104 plt.ylabel('Im(z)')
105 plt.tight_layout()
106 plt.show()

```

Listing 7: Newton Fractal Generator of $z^8 - 1$

5.4 Gallery of Newton Fractals

- $p(z) = z^3 - 1$: Three-fold symmetry, muted blues, reds, greens
- $p(z) = z^4 - 1$: Four-fold symmetry, spiraling tendrils
- $p(z) = z^5 - 1$: Five quadrants with intricate cross-boundary detail
- $p(z) = z^8 - 1$: Eight-fold symmetry, extraordinarily fine structure

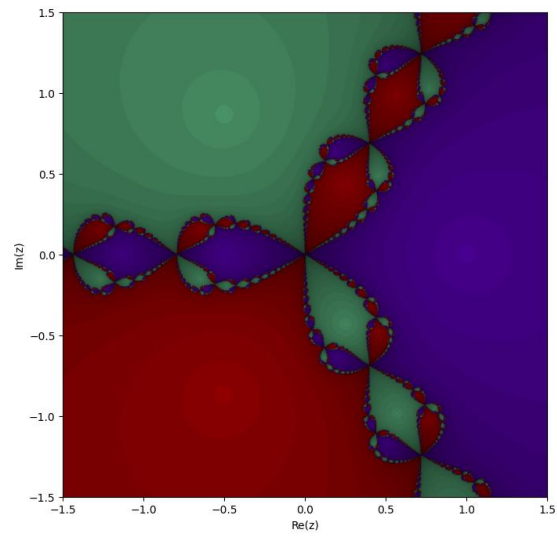


Figure 3: Fractal: $p(z) = z^3 - 1$

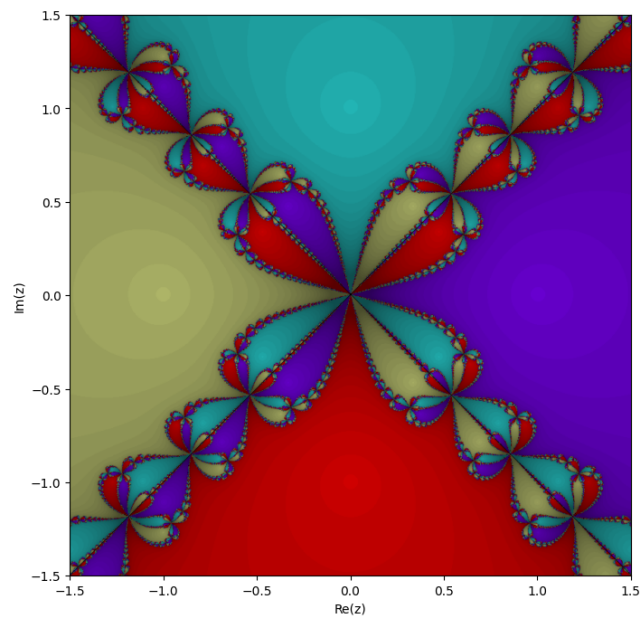


Figure 4: Fractal: $p(z) = z^4 - 1$

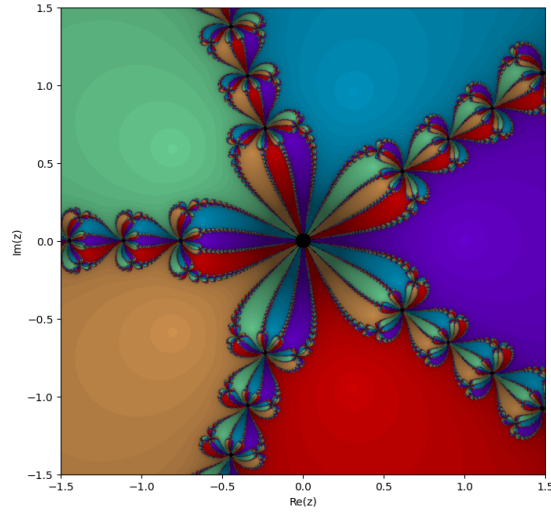


Figure 5: Fractal: $p(z) = z^5 - 1$

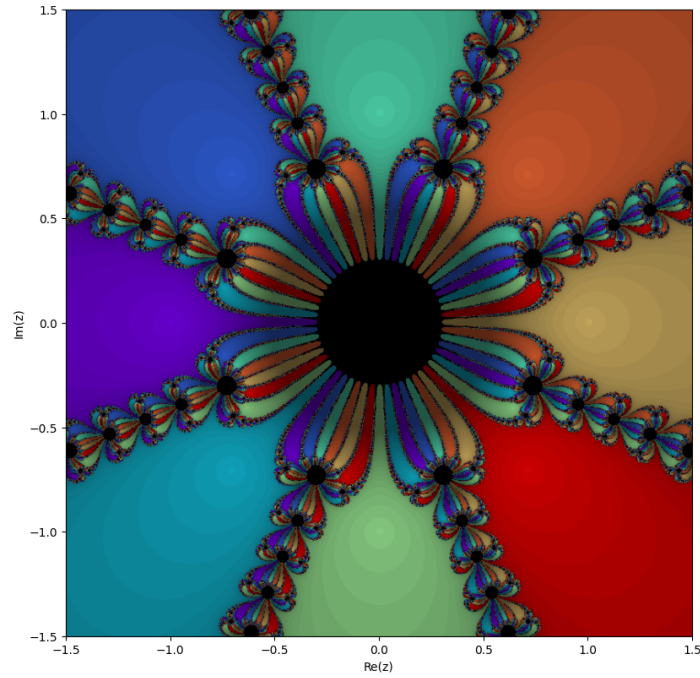


Figure 6: Fractal: $p(z) = z^8 - 1$

6 Conclusion

This paper has presented a unified treatment of classical numerical algorithms, tracing a path from elementary root-finding methods to the remarkable visual complexity of Newton Fractals.

The **Bisection Method** offers guaranteed convergence at the cost of linear speed, making it the method of choice when reliability outweighs performance. The **Secant Method** achieves superlinear convergence without requiring a derivative, at the risk of

divergence with poor initialization. **Newton's Method** delivers quadratic convergence near a simple root.

On the interpolation side, **Lagrange** and **Newton's Divided Difference** methods construct the same unique polynomial through $n + 1$ data points, but Newton's form is preferred when data arrives incrementally. **Cubic Hermite Interpolation** goes further by matching both function values and first derivatives at nodes, producing C^1 -smooth curves essential in computer graphics and robotics.

The culminating section on **Newton Fractals** reveals that numerical algorithms can be sources of unexpected mathematical beauty. When Newton's iteration is extended to the complex plane, the boundaries between convergence basins form Julia sets of infinite intricacy a striking reminder that sensitivity to initial conditions is not merely a computational nuisance, but a deep geometric phenomenon. The analogy to loss landscapes in neural network training is particularly compelling: just as fractal boundaries explain why Newton's method may converge to unexpected roots, non-convex loss surfaces explain why gradient-based optimizers can get trapped in local minima depending on initialization.

Future directions include interactive fractal visualizations with real-time zoom and parameter control, extension to systems of nonlinear equations via multivariate Newton's Method, and a formal comparison of convergence rates across all methods presented here.

References

- [1] Wikipedia contributors. Newton fractal. *Wikipedia, The Free Encyclopedia*, 2024.
- [2] Atkinson, K. E. *An Introduction to Numerical Analysis*, 2nd ed. John Wiley & Sons, New York, 1989.
- [3] Trefethen, L. N. and Bau, D. *Numerical Linear Algebra*. SIAM, Philadelphia, PA, 1997.